

Design Specification

Project Overview

Project Title: TestHub

Description: TestHub is an application designed to collect feedback from users who participate in experiments. Users interact with pre-defined tests, and their responses are recorded and analyzed. The application allows user profile creation, test management, and visual representation of results using charts. Key features include:

- Profile creation and reuse across multiple tests.
- Test results displayed as pie charts.
- Manual addition, editing, and deletion of profiles and tests.
- Tests, users and results are connected with SQLite database.

Use Case Scenario

1. **Scenario:**
 - A user participates in a pre-arranged experiment.
 - The experimenter selects the user's profile or creates a new one.
 - The user completes a test by answering questions.
 - Test results are visualized using charts for immediate feedback.
2. **Storyboard** (Visualized on page 2 of the design document):
 - Profile selection/creation.
 - Test participation.
 - Results analysis with graphical representation.

Cognitive Walkthrough

1. **Scenario:**
 - User participates in some experiment.
 - Experimenter opens the test and chooses the user's profile.
 - User answers the questions.
 - Results are visualized as pie charts.
2. **Steps:**
 - Experimenter opens specified test by selecting test and clicking "Add new result" button.
 - Experimenter chooses user's profile or creates new by clicking "Edit" button.
 - User completes the test.
 - Both of them can see the results as pie graphs.
3. **Walkthrough Table:**

Step	Action	System response
1. Open application	Click on TestHub application	Main window loads with options visible
2. Test creation (if needed)	Click "Manage projects" button	Manage projects window loads with all project options
2.1 Add new test sample	Click "Add" button in Tests menu	Adds new sample test with 1 sample question and 2 sample answers
2.2 Title change	Type new test title Click "Update" button	Test title changes
2.3 Add/remove question	In question menu click "Add" button or select some question and click "Remove" button	Adds question sample with 2 answers or deletes question
2.4 Edit question	Select some question Click "Edit" button in question menu	Manage question window loads with all question options
2.4.1 Question title change	Type new question in specified field	Question title changes
2.4.2 Add/remove answer	In answer menu click "Add" button or select some answer and click "Remove" button	Adds answer sample or deletes selected answer
2.4.3 Edit answer	Select some answer Type new answer text in specified field Click "Update" button in answer menu	Updates selected answer's text
3. User creation (if needed)	Click "Manage users" button	Manage users window loads with all user options
3.1 Add/remove user	Click "Add" button or select some user and click "Remove" button	Adds user sample or deletes selected user
3.2 Add user info	Type required user data in "Login", "Name", "Surname", "Email" fields	Updates user data
4. Result creation	Select some test Click "Add new result" button	Select user window loads with all users to select
4.1 User selection	Select some user Click "OK" button	Test window loads with all test options
4.2 Test completing	Select answers to the questions	Updates result information
4.3 Result as graph	Select some test to see the results as pie graph	Displays test result as pie graph

Technical Implementation

Architecture

1. Technology Stack:

- WPF (Windows Presentation Foundation) using MVVM architecture.
- SQLite database for data storage.

2. Key Functionalities:

- User and test management modules.
- Real-time chart updates using LiveCharts.
- Data persistence and export features.

Modules

1. User Management:

- Create, edit, and delete user profiles.
- Highlight test completion status (red - test is done, green - test is not done).

2. Test Management:

- Create, edit, and delete tests, questions and answers.
- Link questions and answers dynamically to tests.

3. Results Visualization:

- Pie charts displaying user responses for each test.
- Real-time updates based on user interaction.

Requirements

Functional Requirements

1. Allowed to create, edit, and delete user profiles.
2. Support test-taking functionality.
3. Provide graphical representation of test statistics.
4. Enable manual addition, editing, and deletion of tests and questions.
5. Allow test results to be saved and exported.

Non-Functional Requirements

1. Responsive UI optimized for accessibility.
2. Intuitive interface for ease of use.
3. Quick data loading and interactions.

Implementation Highlights

Custom Features

1. **Dynamic UI Elements:**
 - Responsive updates based on user interaction.
2. **Database Management:**
 - Efficient storage and retrieval using SQLite.
 - Unique handling for updates and additions based on user/test status.
3. **Chart Integration:**
 - Used LiveCharts for dynamic pie chart rendering.
 - Integrated user responses into chart visualizations.

Future Improvements

1. Expand user management features with user activity history.
2. Support for additional chart types and advanced analytics.
3. Enable cloud storage for cross-device synchronization.